Assignent #3

[credit Berkeley cd180 assignment 3]

[Auto]Stitching Photo Mosaics

This assignment is all about stitching together many photographs to create larger composite images. There are 2 parts to the assignment:

1. In the **first part**, you will capture source photographs, define correspondences between them, and then warp and composite them.



Part 1: **IMAGE WARPING and MOSAICING**

The goal of Part 1 is to get your hands dirty in different aspects of image warping with a "cool" application -- image mosaicing. You will take two or more photographs and create an image mosaic by registering, projective warping, resampling, and compositing them. Along the way, you will learn how to compute homographies, and how to use them to warp images.

The steps of the assignment are:

- A.1: Shoot and digitize pictures (25 pts)
- A.2: Recover homographies (25 pts)
- A.3: Warp the images (25 pts) [produce at least **two** examples of rectified images]
- A.4: Blend images into a mosaic (25 pts) [show source images and results for **three** mosaics.]

There are some built in functions that are able to do much of what is needed. However, we want you to write your own code. Therefore, you are not allowed to use the following functions in your solution: `cv2.findHomography`, `cv2.warpPerspective`, `cv2.getPerspectiveTransform`, `skimage.transform.ProjectiveTransform`, `skimage.transform.warp`, and similar high-level homography or warping functions. On the other hand, there are a number of very helpful functions (e.g. for solving linear systems, inverting matrices, etc) that you are welcome to use. If there is a question whether a particular function is allowed, ask us.

A.1: Shoot the Pictures

Shoot two or more photographs so that the transforms between them are projective (a.k.a. perspective). The most common way is to fix the center of projection (COP) and rotate your camera while capturing photos.

We expect you to acquire most of the data yourself, but you are free to supplement the photos you take with other sources (e.g. old photographs, scanned images, the Internet).

We're not particular about how you take your pictures or get them into the computer, but we recommend:

- Avoid fisheye lenses or lenses with significant barrel distortion (do straight lines come out straight?). Any focal length is ok in principle, but wide angle lenses often make more interesting mosaics.
- Shoot as close together in time as possible, so your subjects don't move on you, and lighting doesn't change too much (unless you want this effect for artistic reasons).
- Overlap the fields of view significantly. 40% to 70% overlap is recommended. Too little overlap makes registration harder.
- It's OK to vary the zoom (change focal length) between pictures.

Good scenes are: building interiors with lots of detail, inside a canyon or forest, tall waterfalls, panoramas. The mosaic can extend horizontally, vertically, or can tile a sphere. You might want to shoot several such image sets and choose the best.

Deliverables: Show at least 2 sets of images with projective transformations between them (fixed center of projection, rotate camera).

A.2: Recover Homographies

Before you can warp your images into alignment, you need to recover the parameters of the transformation between each pair of images. In our case, the transformation is a homography: **p'=Hp**, where **H** is a 3x3 matrix with 8 degrees of freedom (lower right corner is a scaling factor and can be set to 1). One way to recover the homography is via a set of (p',p) pairs of corresponding points taken from the two images. You will need to write a function of the form:

```
H = computeH(im1_pts,im2_pts)
```

where `im1_pts` and `im2_pts` are n-by-2 matrices holding the (x,y) locations of n point correspondences from the two images and H is the recovered 3x3 homography matrix. In order to compute the entries in the matrix H, you will need to set up a linear system of n equations (i.e. a matrix equation of the form **Ah=b** where h is a vector holding the 8 unknown entries of H). If n=4, the system can be solved using a standard technique. However, with only four points, the homography recovery will be very unstable and prone to noise. Therefore more than 4 correspondences should be provided producing an overdetermined system which should be solved using least-squares.

Establishing point correspondences is a tricky business. An error of a couple of pixels can produce huge changes in the recovered homography. The typical way of providing point matches is with a mouse-clicking interface. You can write your own tool or use online tools like `https://pixspy.com` or any image editing software that displays cursor coordinates (e.g., GIMP, Photoshop).

Deliverables: Implement computeH(im1_pts, im2_pts) function. Show your correspondences visualized on the images, system of equations, and recovered homography matrix.

A.3: Warp the Images

Now that you know the parameters of the homography, you can use this homography to warp each image towards the reference image. Write a function of the form:

```
imwarped_nn = warpImageNearestNeighbor(im,H)
   imwarped_bil = warpImageBilinear(im,H)
```

where `im` is the input image to be warped and H is the homography. Use inverse warping (to avoid holes in the output image).

You must implement **TWO** interpolation methods from scratch (do NOT use `scipy.interpolate.griddata` or similar library functions for interpolation):

- **Nearest Neighbor Interpolation**: Round coordinates to the nearest pixel value
- **Bilinear Interpolation**: Use weighted average of four neighboring pixels

Show results for **both** interpolation methods and compare the quality. Discuss the trade-offs between speed and quality. Some tips from staff:

- There are multiple conventions for where (0,0) is an image: sometimes at the "top-left" of the first pixel, other times at the center of the first pixel. Implementing interpolation may be easier if you consider (0,0) to be the center of the first pixel,

(0.5,0) to be in between the first and second pixel, etc. Loosely related reading: A Pixel Is Not A Little Square (Alv Ray Smith).
- Pay attention to the size of the resulting image. You can predict the bounding box by piping the four corners of the image through your H transform.
- Pay attention to how you treat pixels which don't have any values. Consider using an alpha mask (or alpha channel) here.

Rectification: Once you get this far, you should be able to test all of your code by performing "rectification" on an image. Take a photo containing paintings, posters, or any other known rectangular objects and make one of them rectangular - using a homography. You should do this before proceeding further to make sure your homography/warping is working. Note that since here you only have one image and need to compute a homography for, say, ground plane rectification (rotating the camera to point downward), you will need to define the correspondences using something you know about the image. E.g. if you know that the tiles on the floor are square, you can click on the four corners of a tile and store them in im1_pts while im2_pts you define by hand to be a square, e.g. [0 0; 0 1; 1 0; 1 1]. This is a deliverable.

Deliverables: Implement warpImageNearestNeighbor(im, H) and warpImageBilinear(im, H) using inverse warping. Apply to 2+ images for rectification. Show comparisons.

A.4: Blend the Images into a Mosaic

Warp the images so they're registered and create an image mosaic. Instead of having one picture overwrite the other, which would lead to strong edge artifacts, use weighted averaging. You can leave one image unwarped and warp the other image(s) into its projection, or you can warp all images into a new projection. Likewise, you can either warp all the images at once in one shot, or add them one by one, slowly growing your mosaic.

If you choose the one-shot procedure, you should probably first determine the size of your final mosaic and then warp all your images into that size. That way you will have a stack of images together defining the mosaic. Now you need to blend them together to produce a single image. If you used an alpha channel, you can apply simple feathering (weighted averaging) at every pixel. Setting alpha for each image takes some thought. One suggestion is to set it to 1 at the center of each (unwarped) image and make it fall off linearly until it hits 0 at the edges (or use the distance transform `bwdist`). However, this can produce some strange wedge-like artifacts. You can try minimizing these by using a more sophisticated blending technique, such as a Laplacian pyramid. If your only problem is "ghosting" of high-frequency terms, then a 2-level pyramid should be enough.

If your mosaic spans more than 180 degrees, you'll need to break it into pieces, or else use non-projective mappings, e.g. spherical or cylindrical projection.

Deliverables: Show 3 mosaics with source images. Use weighted averaging or blending to reduce edge artifacts. Explain your procedure.

2. In the **second par**t, you will automate the correspondence-finding portion of Part 1 so that you can create the panoramas automatically. A detailed description of the requirements and scoring for this part are given here.

PART 2: FEATURE MATCHING for AUTOSTITCHING

The goal of this project is to create a system for automatically stitching images into a mosaic. A secondary goal is to learn how to read and implement a research paper. The project will consist of the following steps:
- B.1: Detecting corner features in an image (20 pts)
- B.2: Extracting a Feature Descriptor for each feature point (20 pts)
- B.3: Matching these feature descriptors between two images (20 pts)
- B.4: Use a robust method (RANSAC) to compute and apply homographies (40 pts)
-

For the following sections, we will follow the paper "Multi-Image Matching using Multi-Scale Oriented Patches" by Brown et al. but with several simplifications. Read the paper first and make sure you understand it, then implement the algorithm.

B.1: Harris Corner Detection

- Start with Harris Interest Point Detector (Section 2). We won't worry about muti-scale – just do a single scale. Also, don't worry about sub-pixel accuracy. Re-implementing Harris is a thankless task – so you can use sample code harris.py attached. Include on your webpage a figure of the Harris corners overlaid on the image.
- Implement Adaptive Non-Maximal Suppression (ANMS, Section 3). Include on your webpage a figure of the chosen corners overlaid on the image. This section has multiple moving parts; you may need to read it a few times. You may want to skip this step and come back to it; just choose a random set of corners instead in the meantime.

Deliverables: Show detected corners overlaid on image, with and without ANMS.

B.2: Feature Descriptor Extraction

Implement Feature Descriptor extraction (Section 4 of the paper). Don't worry about rotation-invariance – just extract axis-aligned 8x8 patches. Note that it's extremely important to sample these patches from the larger 40x40 window to have a nice big blurred descriptor. Don't forget to bias/gain-normalize the descriptors. Ignore the wavelet transform section.

Deliverables: Extract normalized 8x8 feature descriptors. Show several extracted features.

B.3: Feature Matching

Implement Feature Matching (Section 5 of the paper). That is, you will need to find pairs of features that look similar and are thus likely to be good matches. For thresholding, use the simpler approach due to Lowe of thresholding on the ratio between the first and the second nearest neighbors. Consult Figure 6b in the paper for picking the threshold. Ignore Section 6 of the paper.

Deliverables: Show matched features between image pairs.

B.4: RANSAC for Robust Homography

For step 4, use 4-point RANSAC as we might discuss later on in class to compute robust homography estimates. At this time, you can look up RANSAC which is an algorithm that removes outliers and implement it. Then, produce mosaics by adapting your code from Part A. You may use the same images from part A, but show both manually and automatically stitched results side by side. Produce at least **three** mosaics.

Deliverables: Implement 4-point RANSAC from scratch. Show comparison of stitching manually and automatically. Create >=3 automatic mosaics.

Please follow submissions as per the first 2 assignments.

SCORING

each part is scored out of 100 as mentioned above for a total of 200.

ACKNOWLEDGEMENTS

This borrowed assignment is also borrows: hybrid images part of this assignment is borrowed from Derek Hoiem's Computational Photography class.